

PgBouncer — менеджер соединений с БД PostgreSQL

При разработке наших приложений для хранения данных используется СУБД PostgreSQL.

Сервер PostgreSQL имеет многопроцессную архитектуру и для каждого соединения с БД поднимает серверный процесс, который обходится довольно дорого. Количество соединений ограничено ресурсами сервера. Если число соединений превышает предел, могут возникнуть ошибки и сбои в работе базы данных, а также в этой ситуации может не хватить соединений другим процессам, хотя большая часть ранее созданных соединений может фактически простаивать в данный момент.

Чтобы избавиться от необходимости каждый раз создавать дорогое соединение с PostgreSQL, используются **менеджеры соединений**. Приложение создает соединение с менеджером, а менеджер пользуется уже существующим соединением с БД.

В нашей компании используется менеджер соединений **pgBouncer**. Он позволяет приложениям многократно использовать ранее созданные соединения с БД. Такой подход оптимизирует работу приложения и экономит ресурсы сервера БД.

Для PgBouncer предусмотрено несколько режимов управления, которые определяют правила повторного использования другими клиентами ранее созданных соединений с БД. Режим управления устанавливаются в опции `pool_mode`. Список режимов:

- **session**. При начале сессии клиенту выделяется серверное соединение, которое приписано клиенту в течение всей сессии и возвращается в пул только после отсоединения клиента.
- **transaction**. Серверное соединение возвращается в пул после завершения транзакции клиента.
- **statement**. Серверное соединение возвращается в пул сразу после завершения запроса. Транзакции с несколькими запросами в этом режиме не разрешены, так как они гарантировано будут отменены, а также не работают подготовленные выражения (prepared statements).

В компании Тензор установлено использовать режим [transaction](#).

Использование pgBouncer в транзакционном режиме

Преимущества и недостатки

В транзакционном режиме серверное соединение БД сопоставляется с клиентским только на время транзакции. Как только pgBouncer понимает, что клиент завершил транзакцию, он возвращает серверное соединение в пул. После этого данное серверное соединение может быть сопоставлено с другим клиентским соединением.

Такой режим имеет большие преимущества по сравнению с сессионным, но накладывает определенные ограничения на функционал, который может использоваться клиентским приложением.

Преимущества

В сессионном режиме серверное соединение с БД захватывается при подключении клиента к pgBouncer'у, а его возврат в пул – при отключении клиента. Обычно в наших приложениях захват соединения происходит при входе в метод, а освобождение – при выходе из него.

Существует множество методов, в которых работа с БД занимает лишь малую часть (например, если метод обращается к другому сервису и ожидает ответа от него). Выполнение таких методов с использованием pgBouncer'a в сессионном режиме приводит к избыточным расходам ресурсов БД.

Транзакционный режим практически полностью лишен таких недостатков. Серверное соединение захватывается лишь на время транзакции, поэтому если приложение выполняет какую-то логику, не используя БД, то оно и не занимает серверного соединения.

Недостатки

В PostgreSQL есть функционал, завязанный на сессию соединения сервера БД с клиентом. В транзакционном режиме накладываются некоторые ограничения на использование такого функционала.

Ограничения

В транзакционном режиме не работает (или работает некорректно) сессионный функционал. Ниже будут перечислены возможности PostgreSQL, недоступные для использования при работе через pgBouncer в транзакционном режиме.

Advisory-блокировки

Время жизни сессионных блокировок ограничено временем жизни серверного соединения. Поэтому если создать сессионную блокировку через pgBouncer, работающий в транзакционном режиме, то возможны проблемы.

Рассмотрим пример: в коде создается advisory-блокировка, соответствующая какой-то записи таблицы, затем данная запись модифицируется, после чего блокировка снимается. Такой код мог бы использоваться (если бы pgBouncer работал в сессионном режиме) для предотвращения одновременного доступа к записи:

- Первый запрос:

```
select pg_advisory_lock('Таблица1'::regclass::integer, 123)
```

- Второй запрос:

```
update "Таблица1" set "Value" = 555 where "@Таблица1" = 123
```

- Третий запрос:

```
select pg_advisory_unlock('Таблица1'::regclass::integer, 123)
```

Все запросы выполняются вне транзакции, поэтому pgBouncer для их выполнения может выделить из пула разные серверные соединения. Серверное соединение, содержащее блокировку, может быть отдано другому клиенту, который хочет получить доступ к этой записи → нарушена логика приложения, так как не предотвращен одновременный доступ к записи от нескольких клиентов.

Также в таком случае с большой долей вероятности произойдет "утечка" блокировки, так как запрос освобождения блокировки может выполняться из соединения, которому блокировка не принадлежит (в таком случае PostgreSQL выдаст в лог предупреждение и проигнорирует запрос; блокировка не будет удалена).

Таким образом, нельзя пользоваться следующими

функциями: **pg_advisory_lock**, **pg_advisory_lock_shared**, **pg_advisory_unlock**, **pg_advisory_unlock_all** и **pg_advisory_unlock_shared**.

Функционирование транзакционных advisory-блокировок не будет нарушено, то есть пользоваться функциями **pg_advisory_xact** можно.

Переменные сессии

В транзакционном режиме pgBouncer может передать серверное соединение с установленными переменными сессиями другому клиенту, а также сопоставить вашему соединению серверное соединение без переменных сессии. Поэтому нельзя пользоваться следующим функционалом:

- **set_config** с параметром **local=false**, например:

```
set_config('sbis3.test', 'test', false)
```

- установка переменных запросом вида:

```
set sbis3.test = 'test'
```

Примечание: переменные транзакции использовать можно. То есть вполне допускается следующие вызовы:

- **set_config** с параметром **local=true**, например:

```
set_config('sbis3.test', 'test', true)
```

- установка локальных переменных транзакции запросом вида (обратите внимание на слово "local" в запросе):

```
set local sbis3.test = 'test'
```

Для установки переменных транзакции в Wasaby Framework реализован специальный механизм, в который можно зарегистрировать собственную callback-функцию. Эта функция будет вызываться перед выполнением каждого метода.

Пример использования:

```
1 // callback, который устанавливает в переменную транзакции
  "sbis3.RandomValue" случайное число
2 dbi::TransactionVariablesSetter::TransactionVariablesVector
  GetRandValue( const dbi::ConnectionParams& )
3 {
4     dbi::TransactionVariablesSetter::TransactionVariablesVector result;
5     result.push_back( std::make_pair( "sbis3.RandomValue",
    boost::lexical_cast<std::string>( rand() ) ) );
6     return result;
7 }
8 static dbi::TransactionVariablesSetter::Registrar reg( GetRandValue
  );
```

Контроль работоспособности сервера бизнес-логики при использовании pgBouncer

Возможные проблемы

При разворачивании системы, использующей pgBouncer для работы с БД, могут возникнуть проблемы, которые можно разделить на несколько групп:

- Ошибки настройки pgBouncer;
- Проблемы, возникающие при работе бизнес логики с БД.

Диагностика проблем настройки pgBouncer

Для того чтобы убедиться, что pgBouncer настроен правильно нужно:

- Зайти в БД через pgBouncer под пользователем, который используется сервисом бизнес логики (можно использовать pgAdmin или любой другой клиент postgresql). Если нужная БД не сконфигурирована в pgBouncer или задан не тот пользователь, то будет выведена ошибка. В нормальной ситуации вход должен пройти успешно.
- Убедиться, что pgBouncer пробрасывает на нужную базу. Для этого зайти в БД напрямую и выполнить следующий запрос:

```
select '"Пользователь"'::regclass::integer
```

Вместо "Пользователь" можно подставить название любой из таблиц, существующих в БД. В результате будет выведен идентификатор этой таблицы. Далее нужно повторить эти действия, подключившись через pgBouncer. Если полученные идентификаторы не совпадают, то это означает, что pgBouncer работает некорректно. В таком случае нужно проверить настройки этой БД в файле конфигурации pgBouncer.ini.

- Посмотреть логи pgBouncer'a. В них не должно быть ошибок.
- Подключиться к pgBouncer'у, посмотреть, что в логах появилась информация о новом подключении.

```
2012-11-12 16:51:35.434 1968 LOG C-01bd6778:
my.test.db/postgres@127.0.0.1:49773 login attempt: db=operator
user=postgres
```

Если в пуле серверных соединений не было свободного соединения, то будет создано новое соединение:

```
2012-11-12 16:11:18.645 1968 LOG S-01bf3960:
my.test.db/postgres@127.0.0.1:5432 new connection to server
```

- Отключиться от pgBouncer. Убедиться, что в логах появилось сообщение о корректном отключении клиента:

```
2012-11-12 16:11:19.035 1968 LOG C-01bd6778:
my.test.db/postgres@127.0.0.1:64874 closing because: client close request
```

(age=0)

- Убедиться, что pgBouncer закрывает старые серверные соединения (соединения, время жизни которых превысило значение, указанное в параметре server_lifetime; обычно это 60 секунд). Для этого достаточно подключиться к pgBouncer'у, чтобы он создал новое серверное соединение, отключиться от него и подождать. В логах должно появиться сообщение следующего вида:

```
2012-11-26 17:35:48.343 9424 LOG S-01cf38e0: my.test.db
/postgres@127.0.0.1:5432 closing because: server lifetime over (age=60)
```

Диагностика проблем сервисов бизнес-логики, запущенных через pgBouncer

- Для того чтобы убедиться, что сервисы бизнес-логики не сломались из-за pgBouncer'a, достаточно проверить основные операции на БД: чтение, обновление и удаление записи. Для этого достаточно зайти в online.sbis.ru и создать какой-нибудь документ (например, счет-фактуру), открыть его, и попробовать удалить. При этом необходимо контролировать, что sbis-гpc-service300.dll не кидает ошибки (для этого можно использовать FireBug). Если вышеописанные запросы сервис бизнес-логики обработал корректно, то это означает, что не возникло проблем при работе с БД.
- Убедиться, что в логах pgBouncer нет критичных ошибок.
- По логам pgBouncer убедиться, что сервисы бизнес-логики подключаются перед выполнением метода и отключаются после его выполнения.
- По логам pgBouncer убедиться, что закрываются старые серверные соединения. В логах должны встречаться строки следующего вида:

```
2012-11-26 17:35:48.343 9424 LOG S-01cf38e0: my.test.db
/postgres@127.0.0.1:5432 closing because: server lifetime over (age=60)
```

Администрирование и получение статистики работы pgBouncer

Диагностика работы pgBouncer

Для получения статистики и администрирования pgBouncer имеет возможность подключения к виртуально БД "pgbouncer" под специальными пользователями (пользователи stats и admin, указанные в файле конфигурации). С помощью специальных команд можно получить статистику работы pgBouncer, а так же изменить его конфигурацию. Диагностический интерфейс pgBouncer'a описан в [статье](#).

Общая статистика баз данных

Команда для получения статистики:

```
SHOW STATS
```

Выдает общую статистику по каждой из баз, настроенных в pgBouncer'e.

Поле	Описание
Database	Statistics are presented per database.
total_requests	Total number of SQL requests pooled by pgbouncer.
total_received	Total volume in bytes of network traffic received by pgbouncer.
total_sent	Total volume in bytes of network traffic sent by pgbouncer.
total_query_time	Total number of microseconds spent by pgbouncer when actively connected to PostgreSQL.

avg_req	Average requests per second in last stat period.
avg_recv	Average received (from clients) bytes per second.
avg_sent	Average sent (to clients) bytes per second.
avg_query	Average query duration in microseconds.

Информация о серверных соединениях pgBouncer

Команда для получения статистики:

```
SHOW SERVERS
```

Поле	Описание
Type	S, for server
user	Username pgbouncer uses to connect to server.
database	Database name on server.
state	State of the pgbouncer server connection, one of active, used or idle.
addr	IP address of PostgreSQL server.
port	Port of PostgreSQL server.
local_addr	Connection start address on local machine.
local_port	Connection start port on local machine.
connect_time	When the connection was made.
request_time	When last request was issued.
ptr	Address of internal object for this connection. Used as unique ID.
link	Address of client connection the server is paired with.

Примечание: В таблице зачеркнуты те пункты, которые не отображаются в сервисе ["Управление облаком"](#) (Анализ статистики/Анализ работы → Прочее → Статистика мультитемпораторов).

Информация о клиентских соединениях pgBouncer

Команда получения статистики:

```
SHOW CLIENTS
```

Поле	Описание
Type	S, for server
user	Client connected user.
database	Database name.
state	State of the client connection, one of active, used, waiting or idle.
addr	IP address of client.
port	Port client is connected to.

local_addr	Connection end address on local machine.
local_port	Connection end port on local machine.
connect_time	Timestamp of later client connection.
request_time	Timestamp of later client request.
ptr	Address of internal object for this connection. Used as unique ID.
link	Address of server connection the client is paired with.

Примечание: В таблице зачеркнуты те пункты, которые не отображаются в сервисе ["Управление облаком"](#) (Анализ статистики/Анализ работы → Прочее → Статистика мультиплексоров).

Информация о состоянии пула соединений

Команда получения статистики:

```
SHOW POOLS
```

Поле	Описание
user	User name.
database	Database name.
cl_active	Count of currently active client connections.
cl_waiting	Count of currently waiting client connections.
sv_active	Count of currently active server connections.
sv_idle	Count of currently idle server connections.
sv_used	Count of currently used server connections.
sv_tested	Count of currently tested server connections.
sv_login	Count of server connections currently login to PostgreSQL.
Maxwait	How long has first (oldest) client in queue waited, in second. If this start increasing, then current pool of servers does not handle requests quick enough. Reason may be either overloaded server or just too small pool_size.

Информация о состоянии pgBouncer'a

Команда для получения:

```
SHOW LISTS
```

Выдает одну запись, в которой указана общая статистика pgBouncer'a.

Поле	Описание
databases	Count of databases.
users	Count of users.
pools	Count of pools.
free_clients	Count of free clients.
used_clients	Count of used clients.

login_clients	Count of clients in login state.
free_servers	Count of free servers.
used_servers	Count of used servers.

Список пользователей, известных pgBouncer'у

Команда для получения:

```
SHOW USERS
```

Выдает список пользователей, прописанных в конфигурации pgBouncer'a.

Информация о базах данных, известных pgBouncer'у

Команда для получения:

```
SHOW DATABASES
```

Возвращает список баз данных, прописанных в конфигурации pgBouncer'a.

Поле	Описание
name	Name of configured database entry.
host	Host pgbouncer connects to.
port	Port pgbouncer connects to.
database	Actual database name pgbouncer connects to.
force_user	When user is part of the connection string, the connection between pgbouncer and PostgreSQL is forced to the given user, whatever the client user.
pool_size	Maximum number of server connections.

Чтение/изменение конфигурации

Команда для получения:

```
SHOW LISTS
```

Возвращает список параметров и их значения, а так же флаги, указывающие, какие из параметров можно модифицировать "на лету".

Поле	Описание
key	Configuration variable name.
value	Configures value.
changeable	Either yes or no, shows if the variable is changeable when running. If no, the variable can be changed only boot-time.

Изменение параметра конфигурации:

```
SET {PARAM_NAME} = {VALUE}
```

Пример:

```
SET min_pool_size = 10
```

Для изменения конфигурации требуется права администратора. Настройки, изменяемые этой командой не сохраняются в файл конфигурации, следовательно, при перезагрузке конфигурации из файла изменения будут утеряны.

Команды для администрирования pgBouncer

Имеется возможность перезапуска, остановки, приостановки, перезагрузки конфигурации.

Поле	Описание
PAUSE	PgBouncer tries to disconnect from all servers, first waiting for all queries to complete. The command will not return before all is done. To be used at the time of database restart.
SUSPEND	All socket buffers are flushed and PgBouncer stops listening data on them. The command will not return before all is done. To be used at the time of PgBouncer restart.
RESUME	Resume work from previous PAUSE or SUSPEND command.
SHUTDOWN	The PgBouncer process will exit.
RELOAD	The PgBouncer process will reload its configuration file and update changeable settings.

Примечание: В таблице зачеркнуты те пункты, которые не отображаются в сервисе ["Управление облаком"](#) (Анализ статистики/Анализ работы → Прочее → Статистика мультиплексоров).

Еще что-то непонятное

- SHOW FDS — Shows list of fds in use. When the connected user has username pgbouncer, connects thru unix socket and has same UID as running process, the actual fds are passed over connection. This mechanism is used to do online restart. Note: The windows environment is not supported. (требуется админские права)

Эффективность pgBouncer в системе ЭДО

pgBouncer способен работать в нескольких режимах: session, transaction и statement.

Для обеспечения возможности работы СБИС через pgBouncer в режиме session потребовались незначительные доработки.

Режим transaction является более ограниченным по сравнению с session:

- Не поддерживаются переменные сессии;
- Нет возможности работы с advisory-блокировками;

Из-за этих ограничений появляется необходимость существенной доработки некоторых давно используемых механизмов СБИС, что является нетривиальной работой. Для того чтобы осознать оправданность данной доработки, был проведен тест скорости работы системы доставки документов OSR, сконфигурированной в различных режимах (различные конфигурации работы с БД).

Режим statement не рассматривается, так как он в корне противоречит архитектуре приложений на Wasaby Framework: не поддерживается транзакционность, нет возможности выполнения сложных запросов к БД.

Описание условий проведения теста

Для обеспечения возможности запуска Wasaby Framework через pgBouncer в режиме transaction была сделана доработка, в рамках которой произведен переход от использования переменных сессии к использованию переменных транзакций. Благодаря этой доработке появилась возможность тестирования производительности всех приложений, не использующих advisory-блокировок.

В качестве тестируемого сервиса был выбран сервис, обслуживающий внешний интерфейс ЭДО (ВИ). Такой выбор был сделан по следующим причинам:

- Методы этого сервиса реализуют нетривиальную бизнес-логику, которая на рабочих серверах выполняется довольно продолжительное время и сильно загружает БД.
- В методах этого сервиса выполняется много различных действий. Таким образом, в тесте будет участвовать довольно большая часть функционала ЭДО.
- Для отправки документов на данный сервис имеется программа "СБИС Коннект". Нет необходимости писать утилиту, которая будет эмулировать действия реальных клиентов.
- В данной части функционала не используются advisory-блокировки.

При выполнении теста сервис был настроен в 16 процессов, "СБИС Коннект" — в 10 потоков. Таким образом, гарантировалось отсутствие очередей на диспетчере сервиса бизнес логики.

Тест был проведен в следующих конфигурациях БД:

1. Сервис обращается напрямую к БД – нет ограничения на использования соединений с БД.
2. Сервис обращается к БД через pgBouncer, работающий в режиме transaction; размер пула 5. Размер пула в 2 раза меньше, чем количество одновременно работающих клиентов, следовательно, за каждое из соединений конкурируют 2 процесса.
3. Сервис обращается к БД через pgBouncer, работающий в режиме transaction; размер пула 35. То есть данном тесте гарантируется, что процессы не конкурируют друг с другом за соединения к БД
4. Сервис обращается к БД через pgBouncer, работающий в режиме transaction; размер пула 1. На основании результатов данного теста можно будет сделать выводы о том, какую часть времени выполнения метода соединение с БД простаивает.
5. Сервис обращается к БД через pgBouncer, работающий в режиме session; размер пула 5.
6. Сервис обращается к БД через pgBouncer, работающий в режиме session; размер пула 35.

Во всех тестах, где pgBouncer работал в transaction режиме, было отключено отсоединение сервиса БЛ от pgBouncer:

- КоличествоВызововПередРассоединением=0
- ВыполнятьРассоединениеССерверомБД=Нет

В тестах, где pgBouncer работал в session режиме, было включено отсоединение сервиса БЛ от pgBouncer после выполнения каждого метода:

- КоличествоВызововПередРассоединением=1
- ВыполнятьРассоединениеССерверомБД=Да

В тесте, где pgBouncer не использовался, было настроено, чтобы сервис БЛ отключался от БД после выполнения 100 методов (конфигурация как на рабочем сервере):

- КоличествоВызововПередРассоединением=100
- ВыполнятьРассоединениеССерверомБД=Да

Результаты теста

В таблице, приведенной ниже, представлены результаты замера времени выполнения метода "ВИ.ОтправитьПакетДокументов", который вызывает СБИС Коннект для отправки пакета документов.

№ теста	Режим (количество соединений БД/ количество потоков)	Среднее время выполнения метода, мс	Минимальное время выполнения метода, мс	Максимальное время выполнения метода, мс
1.	Напрямую к БД (inf/10)	2251.843	1419	7019
2.	Transaction (5/10)	2443.486	1310	5397
3.	Transaction;(35/10)	1899.986	1533	2976
4.	Transaction; (1/10)	13264.684	1489	21639
5.	Session; (5/10)	4377.351	2355	10202
6.	Session; (35/10)	3146.268	2308	5475

Выводы

На основании результатов можно сделать следующие выводы:

- pgBouncer в транзакционном режиме работает значительно быстрее, чем в сессионном режиме.

- В сервисе доставки документов ЭДО соединение с БД используется лишь половину времени. Поэтому даже через одно соединение в транзакционном режиме сервис обрабатывает почти в два раза быстрее, чем работал бы через pgBouncer в сессионном режиме (в сессионном режиме методы выполнялись бы последовательно, поэтому время выполнения каждого из них ~ 30 с; в транзакционном режиме методы выполнялись за 13,2 с).
- Уменьшение количества соединений в 2 раза по сравнению с количеством потоков не замедлило работу сервиса. Таким образом, введение pgBouncer в транзакционном режиме позволит уменьшить размер активных подключений к БД примерно в 2 раза.
- Сервис, работающий через pgBouncer в транзакционном режиме, выполнял методы немного быстрее, чем сервис, работающий с БД напрямую. Возможно, это связано с тем, что в первом случае была ниже нагрузка на сервер БД, из-за чего запросы в БД выполнялись быстрее.

Однако данный тест не может охватить все ситуации и, скорее всего, таких же результатов не удастся добиться при использовании pgBouncer'a в транзакционном режиме на всех приложениях. Это связано с тем, что многие сервисы имеют тривиальные методы, действие которых заключается в одном обращении к БД (методы "Записать", "Прочитать" и т.п.). В таких методах соединение используется 100% времени выполнения, поэтому pgBouncer не сможет разделить соединение с другим процессом, следовательно, прироста производительности от pgBouncer не будет.

Конфигурация PgBouncer для соединения с БД через Unix-сокеты (UDS)

Unix-сокеты существуют только в Unix-based ОС, применяется для межпроцессного взаимодействия. Как результат, использование такого сокета для PgBouncer увеличивает производительность сервера БД.

Чтобы настроить PgBouncer на использование Unix-сокета, изменяют конфигурацию файла `/etc/pgbouncer/pgbouncer.ini`. Минимальный рабочий конфиг файла для ОС CentOS6 и CentOS7:

```
1 [databases]
2 * =
3
4 [pgbouncer]
5 listen_addr = *
6 listen_port = 6543
7 pool_mode = transaction
```

Дополнительно требуется изменить конфигурацию PostgreSQL, чтобы PgBouncer смог установить соединение с БД. Подробнее читайте [здесь](#).

Опции конфига

Подробнее о всех параметрах конфига вы можете прочитать [здесь](#).

Секция [database]

В секции `[database]` множество баз данных, которые отличаются только названиями, можно не прописывать и установить значение `"*"`:

```
1 [databases]
2 * =
```

В ином случае для каждой базы данных используется отдельная строка:

```
1 databases]
2 template1 = port=5433 dbname=mybd1
3 template2 = port=5434 dbname=mybd2
```

Параметр `"host"` из секции `[database]` можно не указывать, когда установлено значение параметра `"unix_socket_dir"`. По умолчанию `"unix_socket_dir"` установлено в значение `"/tmp"` (в этой директории находится unix-socket PostgreSQL).

Секция [pgbouncer]

Чтобы узнать, как именно собран PostgreSQL, обратитесь к таблице `"pg_settings"`.

1. `listen_addr`. Устанавливает список адресов, доступных для TCP-соединений.

2. **listen_port**. Порт для TCP-соединений. Используется 6543 порт.

3. **pool_mode**. Используется только значение "transaction".

Дополнительная настройка PostgreSQL. Файл pg_hba.conf

По умолчанию в конфигурации PostgreSQL установлены права для локального подключения, при которых PgBouncer из под пользователя "pgbouncer" не сможет соединиться с БД. Поэтому нужно изменить настройки PostgreSQL.

1. В файле `/var/lib/pgsql/9.4/data/pg_hba.conf` найдите в строку:

```
local all all peer
```

2. Измените строку, если:

- для подключения к БД под пользователем с использованием пароля.

```
local all all md5
```

- для подключения к БД под любым пользователем без использования пароля.

```
local all all trust
```

Конфигурация PgBouncer для серверов OSR ([inside и online.sbis.ru](http://inside.online.sbis.ru))

Конфигурация для online.sbis.ru

Файл `/etc/pgbouncer/pgbouncer.ini` должен иметь следующее содержимое:

```
1 [databases]
2 * = host=127.0.0.1
3
4 [pgbouncer]
5 logfile = /var/log/pgbouncer.log
6 pidfile = /var/run/pgbouncer/pgbouncer.pid
7 listen_addr = *
8 listen_port = 6432
9 auth_file = /etc/pgbouncer/userlist.txt
10 auth_type = md5
11 pool_mode = transaction
12 default_pool_size = 100
13 max_client_conn = 5000
14 ignore_startup_parameters = extra_float_digits
15 reserve_pool_size = 50
16 application_name_add_host = 1
17
18 admin_users = pgbouncer
19 stats_users = pgbouncer, stats
20
21 server_reset_query =
22 server_lifetime = 60
23 server_check_delay = 60
24
25 log_connections = 0
```

```
26 log_disconnections = 0
27 log_pooler_errors = 1
28
29 client_idle_timeout = 0
30 server_idle_timeout = 60
```

Конфигурация для всех остальных приложений

```
1 [databases]
2 * = host=127.0.0.1
3
4 [pgbouncer]
5 logfile = /var/log/pgbouncer.log
6 pidfile = /var/run/pgbouncer/pgbouncer.pid
7 listen_addr = *
8 listen_port = 6432
9 auth_file = /etc/pgbouncer/userlist.txt
10 auth_type = md5
11 pool_mode = transaction
12 default_pool_size = 100
13 max_client_conn = 5000
14 ignore_startup_parameters = extra_float_digits
15 reserve_pool_size = 50
16 application_name_add_host = 1
17
18 admin_users = pgbouncer
19 stats_users = pgbouncer, stats
20
21 server_reset_query =
22 server_lifetime = 1800
23 server_check_delay = 300
24
25 log_connections = 0
26 log_disconnections = 0
27 log_pooler_errors = 1
28
29 client_idle_timeout = 0
30 server_idle_timeout = 300
```

Файл userlist.txt

В файле "/etc/pgbouncer/userlist.txt" содержится список пользователей с паролями, которых будет пускать pgBouncer. В данном списке должны присутствовать все пользователи, из-под которых будут работать сервисы с БД.

Также в данном файле обязательно должен присутствовать служебный пользователь stats с паролем stats. Для этого в файле должна присутствовать строка:

```
"stats" "stats"
```

Данный пользователь используется только для просмотра состояния и статистики работы pgBouncer. Ему будут доступны лишь эти данные, в БД пользователь зайти не сможет, так как в postgres его нет.

Пример файла "userlist.txt". Первая строка обязательная (см. выше). Остальные строки должны быть заменены на данные о реальных пользователях:

```
1 "stats" "stats"
2 "my_db_user" "mypassword"
```

3 | "user2" "user2_password"